

Difference between ChildProcess close, exit events

Asked 7 years, 1 month ago Modified 6 years, 7 months ago Viewed 28k times



When spawning child processes via `spawn()/exec()/...` in Node.js, there is a `'close'` and an `'exit'` event on child processes.

58

What is the difference between those two and when do you need to use what?



node.js

event-handling

child-process



Share Improve this question Follow

asked May 30, 2016 at 9:20



Narigo

2,929 3 19 30

2 Answers

Sorted by:

Highest score (default)



Before Node.js 0.7.7, there was only an "exit" event on child processes (and no "close" event). This event would be fired when the child process has exited, and all streams (stdin, stdout, stderr) were closed.

64



In Node 0.7.7, the "close" event was introduced ([see commit](#)). The [documentation \(permalink\)](#) currently says:



The 'close' event is emitted when the stdio streams of a child process have been closed. This is distinct from the 'exit' event, since multiple processes might share the same stdio streams.



If you just spawn a program and don't do anything special with stdio, the "close" event fires after "exit". The "close" event can be delayed if e.g. the stdout stream is piped to another stream. So that means that the "close" event can be delayed (indefinitely) after the "exit" event. Does this mean that the "close" event is always fired after "exit"? As the examples below show, the answer is no.

So, if you are only interested in the process termination (e.g. because the process holds an exclusive resource), listening for "exit" is sufficient. If you don't care about the program, and only about its input and/or output, use the "close" event.

Experiment: destroy stdio before killing child

Experimentally (in Node.js v7.2.0), I found that if the stdio streams are not used by the child process, that then the "close" event is only fired after the program has exited:

```
// The "sleep" command takes no input and gives no output.
cp = require('child_process').spawn('sleep', ['100']);
cp.on('exit', console.log.bind(console, 'exited'));
cp.on('close', console.log.bind(console, 'closed'));
cp.stdin.end();
cp.stdout.destroy();
cp.stderr.destroy();
console.log('Closed all stdio');
setTimeout(function() {
  console.log('Going to kill');
  cp.kill();
}, 500);
```

The above program spawning "sleep" outputs:

```
Closed all stdio
Going to kill
exited null SIGTERM
closed null SIGTERM
```

When I change the first lines to a program that only outputs,

```
// The "yes" command continuously outputs lines with "y"
cp = require('child_process').spawn('yes');
```

... then the output is:

```
Closed all stdio
exited 1 null
closed 1 null
Going to kill
```

Similarly when I change spawn a program that only reads from stdin,

```
// Keeps reading from stdin.
cp = require('child_process').spawn('node', ['-e', 'process.stdin.resume()]);
```

Or when I read from stdin and output to stdout,

```
// "cat" without arguments reads from stdin, and outputs to stdout
cp = require('child_process').spawn('cat');
```

Experiment: Pipe program to another, kill first program

The previous experiment is quite artificial. The next experiment is a bit more realistic: You pipe a program to another and kill the first one.

```
// Reads from stdin, output the input to stdout, repeat.
cp = require('child_process').spawn('bash', ['-c', 'while read x ; do echo "$x" ; done']);
cp.on('exit', console.log.bind(console, 'exited'));
cp.on('close', console.log.bind(console, 'closed'));

cpNext = require('child_process').spawn('cat');
cp.stdout.pipe(cpNext.stdin);

setTimeout(function() {
  // Let's assume that it has started. Now kill it.
  cp.kill();
  console.log('Called kill()');
}, 500);
```

Output:

```
Called kill()
exited null SIGTERM
closed null SIGTERM
```

Similarly when the first program only reads from input and never outputs:

```
// Keeps reading from stdin, never outputs.
cp = require('child_process').spawn('bash', ['-c', 'while read ; do : ; done']);
```

When the first program keeps outputting without waiting for stdin, the behavior is different though, as the next experiment shows.

Experiment: Pipe program with lots of output to another, kill first program

```
// Equivalent to "yes | cat".
cp = require('child_process').spawn('yes');
cp.on('exit', console.log.bind(console, 'exited'));
cp.on('close', console.log.bind(console, 'closed'));

cpNext = require('child_process').spawn('cat');
cp.stdout.pipe(cpNext.stdin);

setTimeout(function() {
  // Let's assume that it has started. Now kill it.
  cp.kill();
  console.log('Called kill()');
  setTimeout(function() {
    console.log('Expecting "exit" to have fired, and not "close"');
    // cpNext.kill();
    // ^ Triggers 'error' event, errno ECONNRESET.
    // ^ and does not fire the 'close' event!
  }, 100);
}, 500);
```

```
// cp.stdout.unpipe(cpNext.stdin);
// ^ Does not appear to have any effect.
// ^ calling cpNext.kill() throws ECONNRESET.
// ^ and does not fire the 'close' event!

cp.stdout.destroy(); // <-- triggers 'close'
cpNext.stdin.destroy();
// ^ Without this, cpNext.kill() throws ECONNRESET.

cpNext.kill();
}, 500);
}, 500);
```

The above program outputs the following and then exits:

```
Called kill()
exited null SIGTERM
Expecting "exit" to have fired, and not "close"
closed null SIGTERM
```

Share Improve this answer Follow

answered Nov 30, 2016 at 21:17



[Rob W](#)

340k ● 83 ● 789 ● 677

FTR the doc states The 'close' event will always emit after 'exit' was already emitted, or 'error' if the child failed to spawn. – [Jonathan Giroux](#) Jul 6, 2022 at 15:39



the short version is, 'exit' emits when the child exits but the stdio are **not yet** closed. 'close' emits when the child has exited **and** its stdios are closed.

5



Besides that they share the same signature.

Share Improve this answer Follow

answered May 30, 2016 at 9:25



[user4466350](#)



this info doesn't seem to gel with @Gilad's answer – [Alexander Mills](#) Oct 12, 2016 at 18:32